

RE-VALUE

Piattaforma per il riuso locale di oggetti

Report Milestone #4 — Deliverable D4

Ingegneria del Software — A.A. 2025/2026

Università degli Studi di Trento

Nome e cognome	Matricola	Account GitHub
Alessandro Gremes	242330	alegrem123
Paolo Sarcletti	246846	Puzzapol
Alessandro Turri	244927	alessandroturri

Indice

1	Sezione Introduttiva	1
1.1	Team Members	1
1.2	Project Idea	1
1.3	Link	1
2	Sezione Generale	1
2.1	Strategia di branching	1
2.2	Definizione di “Done”	2
2.3	Product Backlog	2
3	Sezione Sprint #2	4
3.1	Goal	4
3.2	Sprint 2 Planning	4
3.3	Test Cases	6
3.4	Sprint Review	8
3.5	Product Backlog Refinement	9
3.6	Sprint Retrospective	9
4	Sezione Finale	10
4.1	Diagramma del deploy finale	10
4.2	Note sui servizi esterni	11
4.3	Stack tecnologico	12
4.4	Conclusioni	13
A	Matrice di tracciabilità requisiti	13

1 Sezione Introduttiva

1.1 Team Members

Nome e cognome	Matricola	Account GitHub
Alessandro Gremes	242330	alegrem123
Paolo Sarcletti	246846	Puzzapol
Alessandro Turri	244927	alessandroturri

1.2 Project Idea

RE-VALUE è una piattaforma web e mobile per favorire il riuso locale di oggetti ancora utilizzabili. Un donatore pubblica un annuncio, un acquirente lo prenota e lo scambio fisico viene certificato tramite QR code. Alla validazione il sistema completa atomicamente prenotazione e annuncio, aggiorna i wallet con crediti dinamici calcolati al momento della prenotazione, abilita recensioni, notifiche e reputazione.

1.3 Link

Risorsa	Link / credenziali
Repository GitHub	https://github.com/alegrem123/ReValue
Link ad Apiary	https://alessandrogremes.docs.apiary.io/# ; documentazione API storica del progetto, allineata alla specifica Apiary.
Specifica OpenAPI 3 / Swagger	<code>oas3.yaml</code> nel repository; fonte principale del contratto API machine-readable e importabile in Swagger Editor/SwaggerHub.
Swagger UI interattiva	https://revalue-backend-84jb.onrender.com/api-docs/ ; pagina generata da <code>oas3.yaml</code> per consultare e testare le API.
Backend in produzione	https://revalue-backend-84jb.onrender.com
Frontend in produzione	https://revalue-frontend.onrender.com
Account demo user	<code>demo.acquirente@revalue.local</code> / <code>Demo1234!</code>
Account demo admin	<code>demo.admin@revalue.local</code> / <code>Demo1234!</code>

Gli account demo sono generati tramite il comando `npm run seed:demo` nel backend. L'account utente standard dispone di un wallet iniziale di 160 crediti, così da rendere dimostrabili anche il flusso premi e il riscatto dei coupon. L'account admin serve per accedere alla dashboard amministrativa web; nell'app mobile non abilita privilegi o schermate amministrative dedicate. Il seed crea inoltre altri utenti, annunci, prenotazioni e chat per rendere più popolato lo scenario dimostrativo.

2 Sezione Generale

2.1 Strategia di branching

Sprint 1. Il repository non ha usato una strategia *master-only*: sono stati mantenuti due branch permanenti, `main` per la versione stabile e `preRelease` per l'integrazione. Alcuni commit sono comunque arrivati direttamente su `preRelease`; questa è stata una criticità di processo emersa nella retrospettiva e corretta nello Sprint 2.

Sprint 2. Il team ha reso obbligatori i branch temporanei per feature, bug fix o test prima dell'integrazione su `preRelease`. Il flusso usato è:

`feat/fix/test/<nome> → preRelease → main`

I branch effettivamente presenti e usati a supporto della consegna sono:

- **main** — branch stabile, sempre deployabile in produzione; aggiornato solo tramite merge da **preRelease** con pipeline CI verde.
- **preRelease** — branch di integrazione; le feature vengono qui validate prima del rilascio su **main**.
- **feat/AT-*** — branch settimanali di Alessandro Turri (settimane 4, 5, 6).
- **feat/ag-*** — branch di Alessandro Gremes (admin, email, sicurezza, settimana 5).
- **feat/ps-*** — branch di Paolo Sarcletti (recensioni, chat, frontend, Apiary).
- **fix/*** — branch per bug fix urgenti, es. **fix/ag-priority-0**.

I messaggi di commit seguono in larga parte *Conventional Commits* con scope per modulo, ad esempio `feat(premi): ...`, `fix(admin): ...`, `test(security): ...`, `docs(api): ....`. Alcuni commit di merge o di correzione veloce non sono perfettamente uniformi: sono stati mantenuti per preservare la cronologia reale del lavoro, ma la convenzione è stata usata come regola operativa.

Il CI/CD su GitHub Actions si attiva su push verso **main** e **preRelease**: esegue test backend con coverage, smoke test frontend, smoke test mobile e uno smoke test HTTP sul backend Render. Solo a pipeline verde si procede con il merge su **main**.

2.2 Definizione di “Done”

La seguente definizione è stata adottata dal team a partire da Sprint 1 e rafforzata in Sprint 2 con i punti relativi a CI e deploy.

Una user story si considera **Done** quando:

1. Il contratto API è aggiornato in `oas3.yaml`; `apiary.apib` viene riallineato come versione API Blueprint pubblicabile su Apiary.
 2. Tutti i test automatici passano (`npm test` con 0 failing e copertura stabile).
 3. Nessuna regressione critica è stata introdotta nella suite di regressione esistente.
- S2* La pipeline CI GitHub Actions è verde prima del merge su **main**.
S2 Il backend di produzione supera lo smoke test HTTP post-deploy.

2.3 Product Backlog

Il product backlog completo è riportato nella tabella seguente. Le storie di Sprint 1 e Sprint 2 sono marchiate come *Done*; le storie non ancora implementate rimangono nel backlog come lavoro futuro.

ID	User Story	SP	Prio.	Sprint	Stato
PB-01	Come utente, voglio registrarmi con email e password per accedere alla piattaforma.	3	Alta	S1	Done
PB-02	Come utente, voglio fare login e ricevere un token JWT per autenticarmi nelle richieste.	3	Alta	S1	Done
PB-03	Come donatore, voglio pubblicare un annuncio con oggetto, foto e data di scadenza.	8	Alta	S1	Done

ID	User Story	SP	Prio.	Sprint	Stato
PB-04	Come utente, voglio sfogliare il catalogo pubblico con filtri per categoria, dimensione e materiale.	5	Alta	S1	Done
PB-05	Come acquirente, voglio prenotare un oggetto disponibile.	8	Alta	S1	Done
PB-06	Come donatore, voglio generare un QR code per confermare il ritiro dell'oggetto.	5	Alta	S1	Done
PB-07	Come acquirente, voglio scansionare il QR code per completare lo scambio.	5	Alta	S1	Done
PB-08	Come utente, voglio vedere il mio saldo e lo storico transazioni del wallet.	5	Alta	S1	Done
PB-09	Come utente, voglio accedere alle funzionalità principali tramite il frontend web.	8	Alta	S1	Done
PB-10	Come utente, voglio accedere alle funzionalità principali tramite l'app mobile.	5	Media	S1	Done
PB-11	Come utente, voglio lasciare una recensione al partner dello scambio dopo il completamento.	5	Alta	S2	Done
PB-12	Come utente, voglio inviare messaggi al partner dello scambio tramite la chat integrata.	8	Alta	S2	Done
PB-13	Come utente, voglio ricevere notifiche in-app per eventi rilevanti (prenotazione, scambio, ecc.).	5	Alta	S2	Done
PB-14	Come sistema, voglio inviare push notification su dispositivo mobile tramite Expo.	5	Media	S2	Parziale
PB-15	Come utente, voglio riscattare coupon premio con i crediti accumulati.	5	Media	S2	Done
PB-16	Come utente, voglio segnalare comportamenti scorretti di altri utenti.	3	Media	S2	Done
PB-17	Come admin, voglio visualizzare le statistiche della piattaforma (scambi, crediti, utenti).	5	Alta	S2	Done
PB-18	Come admin, voglio sospendere, bannare in modo reversibile e riabilitare utenti.	5	Alta	S2	Done
PB-19	Come admin, voglio gestire e forzare lo stato degli annunci non conformi.	3	Media	S2	Done
PB-20	Come sistema, voglio applicare hardening sicurezza (Helmet, rate-limit, sanitizzazione).	3	Alta	S2	Done
PB-21	Come team, voglio CI/CD automatizzato su GitHub Actions con test e smoke produzione.	3	Alta	S2	Done
PB-22	Come team, voglio il deploy stabile su Render.com con MongoDB Atlas.	5	Alta	S2	Done

ID	User Story	SP	Prio.	Sprint	Stato
PB-23	Come utente, voglio poter aprire un ticket di supporto dalla piattaforma.	2	Bassa	S2	Parziale API
PB-24	Come utente, voglio accedere con SPID/SSO istituzionale.	8	Bassa	—	Backlog
PB-25	Come utente, voglio visualizzare gli annunci vicini su una mappa interattiva.	5	Bassa	—	Backlog
PB-26	Come utente, voglio ricevere notifiche in tempo reale via WebSocket (senza polling).	5	Bassa	—	Backlog
PB-27	Come utente mobile, voglio una schermata dedicata per i ticket di supporto.	2	Bassa	—	Backlog
PB-28	Come sistema, voglio verificare end-to-end la consegna push su dispositivo reale.	3	Media	—	Backlog
PB-29	Come sistema, voglio validare RNF2 con un load test documentato a 500 utenti concorrenti.	5	Media	—	Backlog

3 Sezione Sprint #2

3.1 Goal

Periodo: 18 maggio 2026 – 7 giugno 2026.

Lo Sprint 2 ha avuto come obiettivo principale l'**estensione della piattaforma con le funzionalità sociali e di moderazione** e il **raggiungimento di un deploy stabile in produzione**. Concretamente: completare recensioni, chat, notifiche in-app, sistema premi e segnalazioni; realizzare il pannello di amministrazione con statistiche e moderazione utenti; irrobustire la sicurezza del backend; automatizzare test e deploy tramite CI/CD.

3.2 Sprint 2 Planning

Sprint 2 Backlog

Il backlog di Sprint 2 corrisponde agli item PB-11—PB-23 del product backlog. La colonna “owner” indica il responsabile prevalente, mentre la colonna dei contributi riporta i casi in cui la feature è stata completata o corretta da più membri.

ID	User Story	SP	Owner prevalente / contributi	Stato
PB-11	Recensioni post-scambio	5	P. Sarcletti;	Done
PB-12	Chat e messaggi tra partecipanti	8	integrazione mobile AT P. Sarcletti; notifiche	Done
PB-13	Notifiche in-app	5	AG; hardening AT A. Gremes; hook	Done
PB-14	Push notification Expo	5	PS/AT AG backend, AT	Parziale
PB-15	Sistema premi / coupon	5	mobile/test A. Turri	Done
PB-16	Segnalazioni utenti	3	A. Turri	Done
PB-17	Dashboard admin — statistiche	5	A. Gremes; grafici/statistiche finali	Done
PB-18	Admin moderazione utenti	5	AT AG frontend iniziale; AT backend	Done
PB-19	Admin gestione annunci	3	atomico/fix A. Turri	Done
PB-20	Hardening sicurezza	3	AT regression/security	Done
PB-21	CI/CD GitHub Actions	3	fixes AG workflow iniziale; AT deploy/smoke	Done
PB-22	Deploy Render.com + MongoDB Atlas	5	Node 22 A. Turri	Done
PB-23	Ticket di supporto	2	backend/API/test; UI dedicata futura	Parziale API
Totale impegnato		57	53 SP Done pieni, 4 SP parziali dichiarati	

- **Alessandro Gremes:** 38 commit nel periodo come `alegrem123` più 5 commit con autore **Alessandro Gremes**; aree principali: `/api/v1`, CORS, Apiary iniziale, email, notifiche, admin dashboard, CI, hardening errori/id.
- **Paolo Sarcletti:** 18 commit nel periodo; aree principali: recensioni backend, test recensioni, QR edge cases, chat search/image/polling/typing, frontend profilo pubblico e recensioni.
- **Alessandro Turri:** 143 commit nel periodo; aree principali: premi/coupon, segnalazioni/malus, mobile, deploy, admin backend, sicurezza finale, test regressione, documentazione D4 e tracciabilità.

Questa distribuzione spiega perché alcune user story sono attribuite a un owner prevalente ma riportano contributi incrociati: il lavoro finale è stato integrato e corretto collettivamente, soprattutto sui moduli admin, sicurezza, notifiche e mobile.

Burndown Chart

Sprint 2 è durato 3 settimane (18 maggio – 7 giugno 2026). La velocità iniziale è stata più bassa a causa della definizione dell'architettura dei moduli Social, poi ha accelerato nelle settimane centrali.

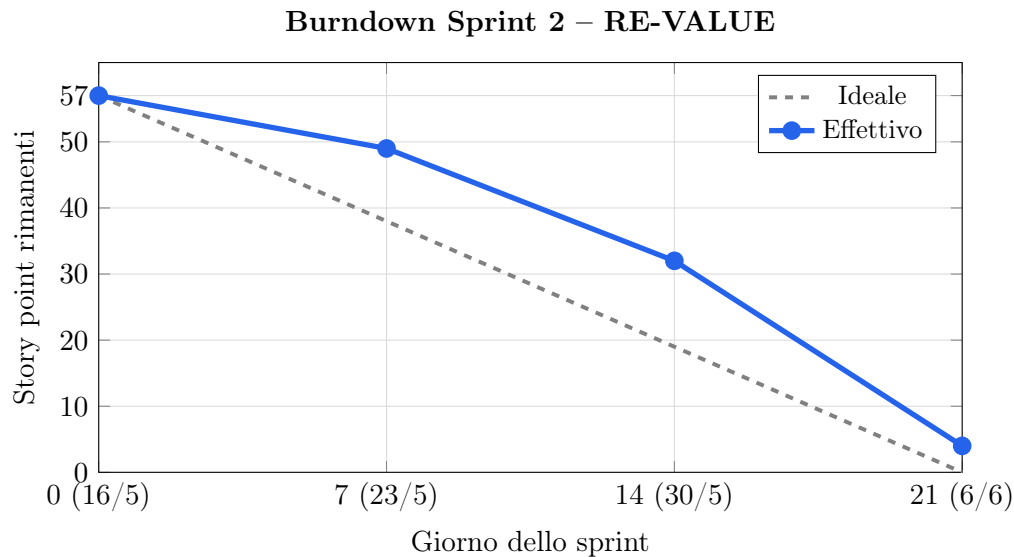


Figura 1: Burndown chart Sprint 2 (57 SP totali; 4 SP residui dichiarati: verifica push nativa su device reale e UI dedicata ticket supporto).

3.3 Test Cases

La verifica finale di Sprint 2 è stata eseguita il **8 giugno 2026** con i comandi: `cd backend && npm test --coverage --coverageReporters=text --runInBand`, `cd frontend && npm test` e `cd mobile && npm test`. L'esito complessivo è: **backend 192/192 test passati su 21 suite**, **frontend 9/9 smoke test passati** e **mobile 11/11 smoke test passati**. La coverage backend rilevata è **69.48% statements**, **57.39% branches**, **67.63% functions** e **71.52% lines**; i model e le route risultano quasi interamente coperti, mentre i controller di chat/messaggi e gli adapter opzionali restano le aree con copertura più bassa.

User Story	N	Descrizione	Dati Test	Precondizioni	Dip.	Risultato Atteso	Ris. Eff.	Note
US01	1	Registrazione valida.	Email valida, password 8+ char.	Email non registrata.	Auth	HTTP 201, JWT, wallet a 0, hash non esposto.	PASS-auth	EP; RF2/RF3, OCL #1-#2.
US01	2	Email già registrata.	Stessa email usata due volte.	Utente esistente.	Auth	Registrazione rifiutata, nessun duplicato.	PASS-auth	EP; unicità email.
US01	3	Password sotto soglia.	Password di 7 caratteri.	Email nuova.	Auth	HTTP 400, vincolo password applicato.	PASS-auth	BVA; RNF4/OCL #2.
US01	4	Login corretto.	Email/password valide.	Utente attivo.	Auth	HTTP 200 con JWT valido.	PASS-auth	EP; RF3.
US01	5	Login con password errata.	Email valida, password errata.	Utente attivo.	Auth	HTTP 401, nessun token.	PASS-auth	EP; credenziali invalide.
US01	6	Login utente bannato.	Credenziali corrette.	bannato=true.	Auth	HTTP 403, accesso negato.	PASS-auth	Cause-effect; moderazione.
US01	7	JWT scaduto.	Token con expiresIn=-1s.	Route privata.	JWT	HTTP 401.	PASS-auth	Error guessing; sessioni.
US01	8	Rate limit login.	6 tentativi stesso identificatore.	Limiter attivo.	Auth	Il sesto tentativo riceve 429.	PASS-auth	BVA; soglia 5.
US02	9	Catalogo pubblico.	GET /api/v1/annunci.	Annunci attivi nel DB.	Ann.	HTTP 200, lista paginata.	PASS-annunci	EP; RF4/UC8.
US02	10	Privacy coordinate anonime.	Richiesta non autenticata.	Annuncio con lat/lng.	Ann.	Coordinate esatte non esposte.	PASS-catalog	Cause-effect; RF25.
US02	11	Filtro categoria.	?categoria=Mobili.	Dataset misto.	Ann.	Solo annunci della categoria richiesta.	PASS-annunci	EP; RF22.
US02	12	Filtro dimensione.	Dimensione grande.	Dataset misto.	Ann.	Solo annunci coerenti.	PASS-annunci	EP; RF22.
US02	13	Filtro materiale.	?materiale=legno.	Dataset misto.	Ann.	Solo annunci coerenti.	PASS-annunci	EP; RF22.
US02	14	Ordinamento per scadenza.	Tre scadenze diverse.	Annunci attivi.	DB	Ordine ASC per dataScadenza.	PASS-annunci	BVA/RNF1.
US03	15	Creazione annuncio valida.	Titolo, oggetto, max 5 foto, scadenza futura.	Donatore autenticato.	Auth	HTTP 201, stato DISPONIBILE.	PASS-annunci	EP; RF15/RF16/OCL #5.
US03	16	Scadenza nel passato.	dataScadenza < now.	Donatore autenticato.	Auth	HTTP 400, annuncio non creato.	PASS-annunci	BVA; OCL #5.

User Story	N	Descrizione	Dati Test	Precondizioni	Dip.	Risultato Atteso	Ris. Eff.	Note
US03	17	Foto oltre limite.	6 foto base64.	Donatore autenticato.	Upload	HTTP 400.	PASS-annunci	BVA; max 5 foto.
US03	18	Campi obbligatori assenti.	Titolo o categoria mancanti.	Donatore autenticato.	Model	HTTP 400.	PASS-annunci	EP; required fields.
US03	19	Modifica annuncio disponibile.	PUT titolo/descrizione.	Donatore, stato DISPONIBILE.	Auth	HTTP 200, dati aggiornati.	PASS-annunci	EP; RF18/OCL #8.
US03	20	Modifica annuncio prenotato.	PUT su stato PRENOTATO.	Donatore proprietario.	Auth	HTTP 409.	PASS-annunci	Cause-effect; OCL #8.
US03	21	Soft delete.	DELETE annuncio disponibile.	Donatore proprietario.	Auth	isAttivo=false, documento conservato.	PASS-annunci	EP; soft delete.
US04	22	Prenotazione valida.	Acquirente prenota annuncio disponibile.	Annuncio attivo e non scaduto.	Auth	HTTP 201, annuncio PRENOTATO.	PASS-swap	EP; OCL #6-#7.
US04	23	Donatore prenota proprio annuncio.	ID annuncio proprio.	Donatore autenticato.	Auth	Prenotazione rifiutata.	PASS-swap	Cause-effect; OCL #4.
US04	24	Prenotazione concorrente.	Due acquirenti simultanei.	Annuncio disponibile.	DB	Un 201, un 409, una sola ATTIVA.	PASS-conc	Cause-effect; OCL #9/RNF2.
US04	25	Stress locale a 10 utenti.	Dieci prenotazioni simultanee.	Annuncio disponibile.	DB	Un successo, nove conflitti.	PASS-conc	Error guessing; concorrenza locale, RNF2 parziale.
US04	26	Annullamento entro finestra.	DELETE prenotazione entro 15 min.	Prenotazione ATTIVA.	Auth	ANNULLATA, annuncio disponibile.	PASS-pren	BVA; OCL #10-#11.
US04	27	Annullamento fuori finestra.	Prenotazione oltre 15 min.	Prenotazione ATTIVA.	Auth	HTTP 409 finestra scaduta.	PASS-pren	BVA; OCL #10.
US05	28	Generazione QR valida.	Donatore chiama /qr/genera.	Prenotazione ATTIVA.	Auth	Token QR non nullo, scadenza futura.	PASS-swap	EP; OCL #13.
US05	29	Acquirente tenta QR.	/qr/genera da acquirente.	Prenotazione attiva.	Auth	HTTP 403.	PASS-qr	Cause-effect; ruoli.
US05	30	Validazione QR valida.	Codice QR corretto.	Prenotazione ATTIVA.	Wallet	COMPLETATA, annuncio RITIRATO, crediti dinamici accreditati a entrambi.	PASS-swap	Cause-effect; OCL #14/RNF5; reverse auction.
US05	31	QR già usato.	Stesso codice due volte.	Scambio completato.	QR	HTTP 409.	PASS-qr	Error guessing; idempotenza.
US05	32	QR scaduto.	Token scaduto.	Documento QR esistente.	QR	HTTP 410.	PASS-qr	BVA; TTL 24h.
US05	33	QR inesistente.	Codice casuale.	Nessun token.	QR	HTTP 404.	PASS-qr	EP; codice invalido.
US06	34	Wallet nuovo.	Creazione utente.	Registrazione valida.	Auth	Bilancio 0, storico vuoto.	PASS-wallet	EP; OCL #15.
US06	35	Accredito positivo.	ammontare=.	Wallet esistente.	Wallet	Bilancio incrementato, transazione registrata.	PASS-wallet	EP; OCL #16.
US06	36	Accredito nullo.	ammontare=0.	Wallet esistente.	Wallet	Errore, saldo invariato.	PASS-wallet	BVA; OCL #16.
US06	37	Sottrazione valida.	Saldo 100, costo 50.	Wallet esistente.	Wallet	Saldo 50, transazione registrata.	PASS-wallet	EP; OCL #17.
US06	38	Saldo insufficiente.	Saldo 10, costo 50.	Wallet esistente.	Wallet	Errore, saldo non negativo.	PASS-wallet	BVA; OCL #15/#17.
US07	39	Chat richiede JWT.	GET messaggi senza token.	Conversazione esistente.	Auth	HTTP 401.	PASS-chat	EP; RF14.
US07	40	Utente estraneo legge chat.	GET da non partecipante.	Conversazione attiva.	Auth	HTTP 403.	PASS-chat	Cause-effect; RF13.
US07	41	Partecipante legge storico.	GET da donatore/acquirente.	Prenotazione attiva.	Auth	HTTP 200 con messaggi.	PASS-chat	EP; RF11.
US07	42	Partecipante invia messaggio.	POST testo valido.	Conversazione attiva.	Auth	HTTP 201, messaggio nello storico.	PASS-chat	EP; RF10.
US07	43	Immagine oltre limite.	POST allegato grande.	Conversazione esistente.	Chat	HTTP 413.	PASS-chat	BVA; allegati.
US08	44	Segnalazione valida.	Motivo non vuoto, target diverso.	Utente autenticato.	Auth	HTTP 201.	PASS-segn	EP; RF9/OCL #18-#19.
US08	45	Auto-segnalazione.	Segnalante = segnalato.	Utente autenticato.	Auth	HTTP 409/400.	PASS-segn	Cause-effect; OCL #19.
US08	46	Motivo vuoto.	Motivo stringa vuota.	Utente autenticato.	Auth	HTTP 400.	PASS-segn	BVA; OCL #18.
US08	47	No-show da acquirente.	POST no-show da acquirente.	Prenotazione attiva.	Auth	HTTP 403.	PASS-pren	Cause-effect; RF19.
US08	48	No-show da donatore.	POST no-show da donatore.	Prenotazione attiva.	Auth	Annullamento, annuncio disponibile, malus +1.	PASS-pren	EP; RF19/OCL #20.
US08	49	Terzo malus.	Applica terzo malus.	Utente con 2 malus.	Admin	Utente auto-sospeso.	PASS-admin	BVA; soglia 3/OCL #20.
US09	50	Lista premi attivi.	GET /premi.	Coupon attivi/non attivi.	Premi	Solo premi attivi ordinati per costo.	PASS-premi	EP; US09.
US09	51	Riscatto saldo sufficiente.	Costo 50, saldo 100.	Utente autenticato.	Wallet	HTTP 201, codice univoco, stock aggiornato.	PASS-premi	EP; OCL #17.
US09	52	Riscatto saldo insufficiente.	Costo 50, saldo 10.	Utente autenticato.	Wallet	HTTP 409, saldo invariato.	PASS-premi	BVA; OCL #17.
US09	53	Riscatto già usato.	PATCH usato due volte.	Riscatto proprietario.	Premi	Secondo update rifiutato.	PASS-premi	Error guessing; stato.
US10	54	Utente normale accede admin.	JWT ruolo user.	Route admin.	Auth	HTTP 403.	PASS-security	EP; ruoli.
US10	55	Sospensione e riabilitazione.	Admin sospende poi riabilita.	Utente non bannato.	Admin	Stato aggiornato, accessi coerenti.	PASS-admin	EP; moderazione.

User Story	N	Descrizione	Dati Test	Precondizioni	Dip.	Risultato Atteso	Ris. Eff.	Note
US10	56	Riabilitazione bannato.	Admin riabilita utente bannato.	bannato=true.	Admin	Blocco rimosso e login nuovamente consentito.	PASS-admin	Cause-effect; decisione S2: ban reversibile.
US10	57	Forzatura stato annuncio.	Admin forza annuncio bloccato.	Prenotazione attiva.	Admin	Stato aggiornato e prenotazioni annullate.	PASS-admin	Error guessing; RF31.
US11	58	Recensione prima dello scambio.	POST su prenotazione attiva.	Scambio non completato.	Auth	HTTP 409.	PASS-review	Cause-effect; OCL #21.
US11	59	Recensione post-scambio.	POST dopo QR valido.	Prenotazione completata.	Auth	HTTP 201.	PASS-review	EP; RF21/RF28.
US11	60	Recensione duplicata.	Stesso autore/prenotazione.	Prima recensione presente.	Auth	HTTP 409.	PASS-review	Error guessing; duplicati.
US12	61	Lista notifiche richiede token.	GET notifiche anonimo.	Notifiche esistenti.	Auth	HTTP 401.	PASS-notif	EP; RF12.
US12	62	Lista notifiche filtrabile.	?letta=false.	Notifiche miste.	Notif.	HTTP 200, paginazione coerente.	PASS-notif	EP; RF12.
US12	63	Marca singola letta.	PATCH notifica propria.	Notifica non letta.	Auth	Solo notifica propria aggiornata.	PASS-notif	Cause-effect; ownership.
US12	64	Token Expo valido.	Token Expo valido.	Utente autenticato.	Mobile	Token salvato nel profilo.	PASS-push	EP; push parziale.
US12	65	Token Expo non valido.	Stringa qualsiasi.	Utente autenticato.	Mobile	HTTP 400.	PASS-push	EP; validazione token.
US13	66	Ticket supporto valido.	Oggetto e messaggio non vuoti.	Utente autenticato.	Supp.	HTTP 201.	PASS-supp	EP; supporto S2.
US13	67	Ticket supporto vuoto.	Messaggio vuoto.	Utente autenticato.	Supp.	HTTP 400.	PASS-supp	BVA; required.
US13	68	Lista ticket personali.	GET ticket utente.	Ticket di più utenti.	Supp.	Solo ticket del richiedente.	PASS-supp	Cause-effect; privacy.
S2-N1	69	Endpoint legacy scambi.	GET/POST /api/v1/scambi/*.	Route legacy.	API	HTTP 410 con Deprecation.	PASS-legacy	Error guessing; compatibilità.
S2-N2	70	Smoke frontend.	Node test su viste web.	Repository completo.	Web	8 smoke test passati.	PASS-front	Regression; frontend.
S2-N3	71	Smoke mobile.	Node test su schermate RN.	Repository completo.	Mobile	10 smoke test passati.	PASS-mobile	Regression; mobile.
S2-N4	72	CI backend con coverage.	GitHub Actions su push/PR.	Branch main/preRelease.	CI	Test backend, frontend e mobile eseguiti.	PASS-CI	White-box; regressione.
S2-N5	73	Push end-to-end device reale.	Device fisico Expo/EAS.	Permessi e build configurati.	Expo	Notifica ricevuta sul device.	PENDING-PB28	Manuale; dipende da EAS.

3.4 Sprint Review

La Sprint Review si è svolta al termine della terza settimana di Sprint 2 (8 giugno 2026). Durante la demo è stato mostrato l'intero flusso della piattaforma in produzione: registrazione, login, pubblicazione annuncio, prenotazione, generazione QR, scansione, completamento scambio con aggiornamento wallet tramite crediti dinamici, invio messaggio in chat, riscatto coupon, invio segnalazione e gestione da pannello admin (statistiche, moderazione utente, forzatura stato annuncio).

Aspetti rilevanti della demo:

- Tutti i flussi principali sono risultati operativi in produzione; la verifica del 8 giugno ha confermato backend 192/192, frontend 9/9 e mobile 11/11 test passati.
- Il pannello admin ha mostrato le statistiche in tempo reale calcolate dalla suite di aggregazioni MongoDB.
- La demo della chat ha evidenziato il funzionamento del middleware `requireParticipant`: utenti estranei non possono accedere allo storico.
- Le notifiche in-app vengono create e segnate come lette correttamente; il canale push su dispositivo reale non è stato dimostrato live per dipendenza dalla configurazione EAS.

Discussione: Nel complesso la demo ha soddisfatto i criteri di accettazione dei principali use case. Le limitazioni emerse e dichiarate nel report sono due: la consegna push nativa non è stata verificata end-to-end su dispositivo fisico con build EAS, e il modulo supporto è completo lato backend/API ma non ha ancora una schermata dedicata comparabile alle altre aree web/mobile. Sono stati predisposti due account demo verificati in produzione per rendere la valutazione ripetibile senza interventi manuali su MongoDB Atlas.

3.5 Product Backlog Refinement

Il meeting di refinement si è tenuto al termine della seconda settimana di Sprint 2. Le principali variazioni al product backlog sono state le seguenti.

- Le storie relative a SPID/SSO e mappa OpenStreetMap (PB-24, PB-25) sono state spostate in icebox: il rapporto tra effort e valore prototipale non giustifica l'integrazione entro la deadline D4.
- La storia PB-14 (push notification nativa su device reale) è stata ridimensionata: il backend Expo Push e il package `expo-notifications` nel client mobile sono presenti; la verifica end-to-end su dispositivo reale (PB-28) è stata separata e spostata nel backlog futuro.
- È stata aggiunta la storia PB-23 (ticket di supporto) dopo il feedback ricevuto durante la Sprint Review intermedia. Entro Sprint 2 sono stati completati backend, API e test; la UI dedicata è stata spostata nel backlog futuro per non ridurre la qualità dei flussi core.
- Il sistema crediti è stato evoluto da accredito fisso a *reverse auction*: i punti assegnati a donatore e acquirente sono calcolati dinamicamente in base a categoria e vicinanza alla scadenza, poi salvati sulla prenotazione e accreditati solo alla validazione QR.
- La moderazione PB-18 è stata interpretata come blocco amministrativo reversibile: l'utente bannato non può accedere finché il flag `bannato=true`, ma un admin può riabilitarlo in caso di errore o revisione manuale. Questa scelta mantiene tracciabilità e recuperabilità dell'account nel prototipo accademico.
- È stato identificato un bug nella logica dello scheduler (annunci scaduti non venivano marcati `isAttivo=false`) e nel metodo `forzaStatoAnnuncio` (non ripristinava `isAttivo=true`): entrambi i fix sono stati inclusi nello sprint corrente come task tecnici.

3.6 Sprint Retrospective

La retrospettiva si è tenuta l'8 giugno 2026, a valle della Sprint Review e della preparazione della documentazione D4.

Cosa ha funzionato:

- La separazione in layer (route → controller → service → model) ha permesso di aggiungere tutti i moduli Sprint 2 senza riscrivere il backend: ogni membro ha lavorato su file distinti con conflitti minimi.
- L'adozione di Conventional Commits e branch per membro ha reso la cronologia git leggibile e ha semplificato la costruzione della lista commit finale.
- La suite di test automatici (backend 192/192, frontend 9/9, mobile 11/11) ha dato fiducia nelle modifiche: ogni merge su `preRelease` era verificato dal CI prima di toccare `main`.
- Il deploy su Render.com ha richiesto meno effort del previsto; la configurazione con variabili d'ambiente ha tenuto le credenziali fuori dal repository.

Cosa non ha funzionato:

- La gestione del time-boxing non è stata sempre rispettata: alcune storie (chat, admin moderazione) si sono rivelate più complesse del previsto e hanno richiesto più SP di quanto stimato.
- L'integrazione push nativa ha richiesto tempo su configurazione EAS che non era prevista inizialmente; il risultato è parziale.
- I bug nello scheduler e in `forzaStatoAnnuncio` sono stati scoperti tardi: un code review più attento sulle funzioni critiche avrebbe permesso di intercettarli prima.

Adozione di pratiche agili: Durante lo sviluppo di RE-VALUE il team ha adottato un processo agile ispirato a Scrum, adattato al contesto accademico e alla dimensione ridotta del gruppo. Il

lavoro è stato organizzato in sprint incrementali, con un backlog mantenuto e aggiornato in base alle priorità funzionali, ai vincoli del corso.

All’inizio dello Sprint 2 il team ha svolto una fase di planning per selezionare le user story più rilevanti: chat, notifiche, recensioni, premi, segnalazioni, moderazione admin, deploy e hardening API. A ciascuna attività sono stati associati story point e un responsabile prevalente, pur mantenendo collaborazione trasversale nei punti più critici, come QR, wallet, sicurezza e documentazione.

La gestione del lavoro è stata supportata da una strategia Gitflow semplificata: **main** come branch stabile, **preRelease** come branch di integrazione e branch temporanei **feat/***, **fix/*** e **test/*** per lo sviluppo delle singole attività. Questa scelta ha permesso di integrare progressivamente le funzionalità e di mantenere una cronologia leggibile tramite Conventional Commits.

Il team ha applicato una Definition of Done condivisa: una user story è stata considerata conclusa solo dopo aggiornamento del contratto API, implementazione, test automatici o smoke test, verifica su ambiente integrato e assenza di regressioni critiche. Per le funzionalità backend più rischiose sono stati aggiunti test di regressione con Jest/Supertest, mentre frontend e mobile sono stati verificati con smoke test automatici e prove manuali sui flussi principali.

Durante lo sprint sono stati svolti momenti frequenti di sincronizzazione informale, usati per riallineare priorità, risolvere blocchi tecnici e ridistribuire attività quando alcune stime iniziali si sono rivelate ottimistiche. La retrospettiva di Sprint 1 ha portato a decisioni concrete in Sprint 2: introduzione sistematica del prefisso **/api/v1**, maggiore attenzione alla documentazione API, rafforzamento dei test su concorrenza e sicurezza, e deploy stabile su cloud.

Il processo non è stato applicato in modo puramente formale: alcune revisioni sono state svolte direttamente tramite integrazione su **preRelease** e controllo con CI, invece che sempre tramite pull request strutturate. Questa deviazione è stata documentata come adattamento pragmatico alla dimensione del team, mantenendo comunque i principi agili di incremento funzionante, ispezione continua, adattamento e trasparenza sulle decisioni tecniche.

Spunti per eventuali sprint futuri:

- Completare la verifica push end-to-end su dispositivo fisico con EAS.
- Eseguire e documentare un load test RNF2 con 500 utenti concorrenti simulati.
- Valutare WebSocket per le notifiche in tempo reale (attualmente il frontend fa polling).
- Aggiungere indice geospaziale 2dsphere per ottimizzare le query di catalogo con filtro distanza.
- Migrare l’hashing password da SHA-256 a bcrypt/Argon2 in un contesto di produzione reale.

4 Sezione Finale

4.1 Diagramma del deploy finale

Il sistema è deployato su un’architettura cloud a tre blocchi principali: repository e pipeline CI/CD su GitHub, servizi applicativi su Render e persistenza su MongoDB Atlas. Il branch **main** rappresenta la versione stabile consegnabile: dopo il merge da **preRelease**, GitHub Actions esegue test backend con coverage, smoke test frontend/mobile e controlli HTTP sugli ambienti di produzione. Solo dopo una pipeline verde il deploy Render viene considerato valido per la consegna.

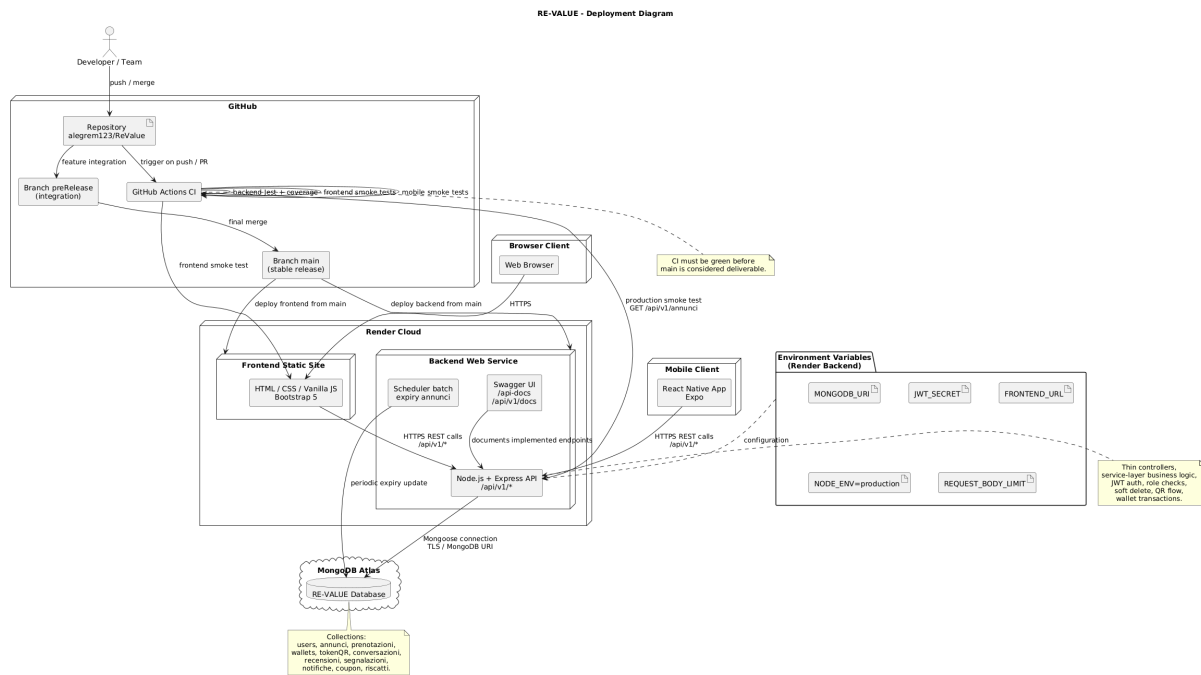


Figura 2: Deployment diagram di RE-VALUE: GitHub Actions, servizi Render, client web/mobile e MongoDB Atlas.

Il backend è eseguito come Web Service Render e contiene l'API Node.js/Express esposta sotto il prefisso `/api/v1`, la Swagger UI accessibile da `/api-docs` e lo scheduler batch per la scadenza automatica degli annunci. La configurazione sensibile è gestita tramite variabili d'ambiente Render, tra cui `MONGODB_URI`, `JWT_SECRET`, `FRONTEND_URL`, `NODE_ENV=production` e `REQUEST_BODY_LIMIT`.

Il frontend web è deployato come sito statico Render e comunica con il backend tramite chiamate HTTPS REST verso gli endpoint versionati. L'app mobile Expo/React Native usa lo stesso contratto API e si collega al backend di produzione tramite HTTPS. La persistenza è affidata a MongoDB Atlas, raggiunto dal backend tramite Mongoose e connection string TLS. Le principali collezioni includono utenti, annunci, prenotazioni, wallet, token QR, conversazioni, recensioni, segnalazioni, notifiche, coupon e riscatti.

Questa configurazione separa chiaramente client, API e database, mantiene le credenziali fuori dal repository e rende verificabile il rilascio tramite CI/CD e smoke test su produzione.

4.2 Note sui servizi esterni

- **SPID/SSO:** non implementato nel prototipo. L'autenticazione usa email/password con JWT. Dichiarato come scelta progettuale per il contesto accademico.
- **OpenStreetMap (backend):** non attivo come servizio backend. La geolocalizzazione rimane lato client; le coordinate degli annunci vengono filtrate via haversine in-memory per il catalogo con raggio.
- **Email (Nodemailer/SMTP):** adapter opzionale. Invia solo se `SMTP_USER` e `SMTP_PASS` sono configurati; salta senza bloccare il flusso applicativo in assenza di configurazione.
- **Expo Push API:** il backend chiama `https://exp.host/-/api/v2/push/send` in modalità fire-and-forget quando l'utente ha un `expoPushToken` valido. Il package `expo-notifications` è presente nel client mobile; la verifica end-to-end su dispositivo fisico con configurazione EAS è stata rimandata al backlog futuro (PB-28).

- **SHA-256 salato per hashing password:** scelta deliberata per il prototipo accademico, in coerenza con il vincolo OCL #2 implementato nel modello utente: il backend accetta hash legacy a 64 caratteri o il formato salato `salt:sha256`. In un contesto produttivo si raccomanda bcrypt o Argon2 con parametri di costo configurabili.

4.3 Stack tecnologico

Backend

- **Runtime:** Node.js 22 (LTS) — versione usata in ambiente CI/CD; lo sviluppo locale è stato condotto su versioni recenti compatibili.
- **Framework:** Express 5.2
- **Database:** MongoDB 7, Mongoose 8 (ODM)
- **Autenticazione:** JWT (jsonwebtoken 9), hashing SHA-256 (crypto nativo)
- **Sicurezza:** Helmet 8, express-rate-limit 8, express-mongo-sanitize 2, CORS configurato per ambiente
- **Email:** Nodemailer 8 (adapter opzionale)
- **Logging:** Morgan 1
- **Testing:** Jest 29, Supertest 7, mongodb-memory-server 9 (MongoMemoryReplSet per transazioni)
- **CI:** GitHub Actions (Node.js 22, coverage obbligatoria)
- **Deploy:** Render.com Web Service

Frontend web

- HTML5, CSS3, JavaScript ES modules (vanilla, nessun framework)
- Bootstrap (layout e componenti UI)
- Chart.js (grafici nella dashboard admin)
- jsQR (decodifica QR da browser)
- Client API centralizzato (`frontend/js/apiClient.js`)
- Deploy: Render.com Static Site

App mobile

- Expo 54, React Native 0.81.5
- @react-native-async-storage/async-storage (persistenza token)
- expo-camera 17 (scansione QR)
- expo-image-picker 17 (foto annunci)
- expo-location 19 (geolocalizzazione)
- expo-notifications 0.32 (push notification, installato, integrazione end-to-end su device reale in PB-28)
- react-native-qrcode-svg 6, react-native-svg 15 (generazione QR)
- @expo/vector-icons 15

Documentazione e strumenti

- OpenAPI 3 / Swagger (oas3.yaml) — fonte formale e machine-readable del contratto API
- API Blueprint / Apiary (apiary.apib) — documentazione pubblicabile su Apiary
- Conventional Commits per la cronologia git
- GitHub Actions per CI/CD con smoke test su produzione

4.4 Conclusioni

RE-VALUE è partito come un nucleo minimale (auth, annunci, prenotazione, QR, wallet) e nel corso dei due sprint si è evoluto in una piattaforma completa con funzionalità sociali, moderazione admin, sistema reputazionale, crediti dinamici e deploy in produzione su cloud. La suite automatica finale passa con 191 test backend su 21 suite, 9 smoke test frontend e 11 smoke test mobile, garantendo un livello di confidenza adeguato per un prototipo accademico.

Le scelte tecnologiche più discusse — SHA-256 per hashing, assenza di SPID/SSO, push nativa non verificata end-to-end su device reale — sono state documentate esplicitamente come decisioni deliberate per il contesto di prototipo, con indicazione delle evoluzioni consigliate per un contesto produttivo.

Dal punto di vista del processo, l'adozione di Gitflow semplificato e CI/CD obbligatoria ha nettamente migliorato la qualità delle integrazioni rispetto a Sprint 1. La comunicazione interna è stata efficace, anche se le stime iniziali degli SP per alcuni moduli (chat, admin) si sono rivelate ottimistiche.

Il team considera gli obiettivi di Sprint 2 sostanzialmente raggiunti: tutte le user story ad alta priorità sono Done, i flussi principali sono verificabili in produzione, la documentazione Apiary/OpenAPI è allineata al backend e il report integra test cases e matrice di tracciabilità.

A Matrice di tracciabilità requisiti

La tabella seguente mappa ogni requisito funzionale (RF), use case (UC) e vincolo OCL ai moduli backend, alle schermate frontend/mobile e alle evidenze di test. Gli stati sono: **Completo**, **Parziale**, **Prototipale**, **Futuro**.

Area	RF/UC/OCL	Backend / API	Frontend / Mobile	Test	Stato
Autenticazione	Auth	authRoutes,authController, jwt.js,authService	login.html,register.html, AuthScreen	auth.test.js, security.test.js	Completo
Password hash	OCL #2 / RNF	password.js,userModel.js	login/register web e mobile	auth.test.js	Prototipale ¹
Profilo utente	Users	usersRoutes,usersController	profile.html,ProfileScreen	smoke	Completo
Profilo pubblico	RF8	GET/users/:id/profilo, recensioniController	public-profile.html	reviews.test.js	Completo
Catalogo	RF4 / UC8 / RF22	GET/annunci, annunciController	catalog.html,CatalogScreen	annunci.test.js, catalog_flow.test.js	Completo
Crea annuncio	RF15 / RF16 / OCL #5	POST/annunci, annuncioModel	create-annuncio.html, CreateAnnuncioScreen	annunci.test.js, catalog_flow.test.js	Completo
Modifica annuncio	RF18 / OCL #7	PUT/annunci/:id	edit-annuncio.html, MyAnnuncioScreen	annunci.test.js	Completo
Cancellazione	RF18 / OCL #8	DELETE/annunci/:id	my-annuncio.html	annunci.test.js, catalog_flow.test.js	Completo
Stato annuncio	state machine	PATCH/annunci/:id/status	viste donatore	annunci.test.js	Completo
Prenotazione	UC2 / OCL #4,#6, #9,#10	prenotazioniRoutes, prenotazioniController, prenotazioniService	annuncio.html, MyBookingsScreen	prenotazioni.test.js, concurrent.test.js	Completo
Concorrenza	OCL #9,#10 / RNF2 parziale	update atomico su campo versione	n/a	concurrent.test.js	Parziale

Area	RF/UC/OCL	Backend / API	Frontend / Mobile	Test	Stato
Annullamento	RF19 / RF20 / OCL #11	disdetta e no-show routes	mybookings.html, MyBookingsScreen	prenotazioni.test.js, prenotazione_flow.test.js	Completo
QR scambio	UC3 / RF17,RF27 / OCL #13-#15	qrRoutes, scambioQrService	qr-display.html, qr-scan.html, QRScanScreen	swap.test.js, qr_edge_cases.test.js	Completo
Wallet	RF5,RF6 / OCL #16,#17	walletRoutes,walletService	wallethistory.html	wallet.test.js, swap.test.js	Completo
Premi / coupon	UC7 / OCL #17	premiController, couponModel,riscattoModel	premi.html,my-premi.html, PremiScreen	premi.test.js	Completo
Recensioni	RF8,RF21,RF28 / OCL #21	recensioniController	swap-success.html, public-profile.html, SwapSuccessScreen	reviews.test.js	Completo
Chat	RF10,RF11, RF13,RF14	chatRoutes,messaggiRoutes, requireParticipant	chat.html,messaggi.html, ChatScreen	chat_access.test.js, smoke	Completo
Notifiche in-app	RF12	notificheService, notificaModel	notifiche.html, NotificheScreen	notifiche.test.js	Completo
Push native	RF12 esteso	sendExpoPushIfConfigured, PATCH/users/me/push-token	token mobile (parziale)	support_push.test.js	Parziale ²
Email Segnalazioni	servizio esterno RF9 / OCL #18,#19	emailService.js segnalazioniController	n/a segnala.html,my-reports.html	doc servizi segnalazioni.test.js, security.test.js	Prototipale Completo
Moderazione admin	RF29 / OCL #20	adminController,userService	admin/dashboard.html	admin_moderation.test.js	Completo
Dashboard admin	RF30,RF31 / UC14	GET/admin/statistiche	dashboard admin	admin_moderation.test.js	Completo
Ticket supporto	S2	supportoController, ticketSupportoModel	nessuna UI dedicata	support_push.test.js	Parziale ³
Contratto API	REST / Apiary	apiary.apib,/api/v1/*	client web/mobile	smoke API client	Completo
App mobile	prototipo	App.js, mobile/src/screens/	React Native/Expo	mobile/tests/smoke	Completo prototipo
CI/CD	processo	.github/workflows/ci.yml	deploy Render	GitHub Actions	Completo
SPID/SSO, OSM	feature future	n/a	n/a	n/a	Futuro

¹SHA-256 è scelta accademica dichiarata; bcrypt/Argon2 consigliati per produzione.

²Backend pronto e testato; verifica end-to-end su device fisico (EAS) rinviata a PB-28.

³Backend implementato e testato; schermata UI mobile dedicata in PB-27.